ОПИСАНИЕ ЗАДАЧИ

Предположим, у нас есть объект, который движется с постоянной скоростью (автомобиль). Мы можем измерять его положение, но эти измерения содержат шум.

Задача — оценить положение объекта, используя фильтр Калмана, который будет сочетать наши измерения с предсказанием на основе скорости объекта.

ОСНОВНЫЕ КОМПОНЕНТЫ

- Состояние объекта: мы будем отслеживать только одно состояние — положение объекта. Скорость в этом примере будем считать известной и постоянной.
- Измерения: Мы можем измерять положение объекта, но эти данные будут содержать шум.
- Шум измерений: В реальной жизни датчики могут давать данные с ошибкой, и это мы учтем в модели.
- Предсказание: Мы предскажем новое положение объекта на основе его предыдущего положения и известной скорости.
- Обновление: Мы скорректируем наше предсказание на основе измерений, чтобы улучшить точность оценки.

+ Код + Текст

Импорт библиотек

```
import numpy as np # Модуль для работы с массивами, мат. операциями, генерации случайных чисел
import matplotlib.pyplot as plt # Библиотека для визуализации, построения графиков
```

Класс реализации фильтра Калмана

Этот класс реализует сам фильтр Калмана. Его задача — обновлять оценку состояния объекта на основе шума измерений и предсказаний.

```
class SimpleKalmanFilter:
    def __init__(self, initial_state, initial_uncertainty, process_variance, measurement_variance):
        self.state = initial_state  # Нынешнее состояние (положение объекта)
        self.uncertainty = initial_uncertainty  # Начальная неопределенность оценки. Чем больше эта неопределенность, тем меньше уверенно
        self.process_variance = process_variance  # Ковариация шума процесса (неопределенность, связанная с ошибками модели, например, ко
        self.measurement_variance = measurement_variance  # Ковариация шума измерений (например, неопределенность датчиков)

    def predict(self, velocity):
        """Предсказание следующего состояния."""
        # Предполагаем, что объект движется с постоянной скоростью
        self.state += velocity  # Положение увеличивается на величину скорости
        self.uncertainty += self.process_variance  # Увеличиваем неопределенность с учетом шума процесса

    def update(self, measurement):
        """Обновление состояния с учетом измерений."""
        # Вычисление Калмановского коэффициента (он говорит, насколько доверять измерениям)
        kalman_gain = self.uncertainty / (self.uncertainty + self.measurement_variance)

        # Обновляем текущее состояние с учетом измерений
        self.state = self.state + kalman_gain * (measurement - self.state)

        # Обновляем неопределенность после получения нового измерения
        self.uncertainty = (1 - kalman_gain) * self.uncertainty

        return self.state, self.uncertainty
```

Метод **predict**

Этот метод предсказывает следующее состояние системы на основе текущего состояния и скорости.

Здесь происходит два шага:

- Мы обновляем текущее положение объекта, добавляя к нему скорость. Это предсказание основано на предположении, что объект движется с постоянной скоростью.
- Мы также увеличиваем неопределенность, добавляя шум процесса, поскольку наша модель может быть неточной.

Метод **update**

Этот метод обновляет оценку состояния на основе поступивших измерений.

- Вычисление Калмановского коэффициента. Он определяет, насколько доверять измерению относительно предыдущей оценки. Чем больше неопределенность текущего состояния по сравнению с шумом измерения, тем больше будет коэффициент Калмана.

Формула в данной задаче:

$$K = \frac{\text{uncertainty}}{\text{uncertainty} + \text{measurement_variance}}$$

Где:

- K — это **Калмановский коэффициент.**
- uncertainty — неопределенность текущего состояния.
- $\text{measurement_variance}$ — дисперсия измерений (ошибка датчиков).

- Обновление состояния. Мы обновляем текущее положение, используя измерение и Калмановский коэффициент. Если коэффициент Калмана близок к 1, то мы больше доверяем измерению, если ближе к 0 — больше доверяем предыдущему состоянию.

Формула в данной задаче:

$$\text{new_state} = \text{old_state} + K \times (\text{measurement} - \text{old_state})$$

Где:

- new_state — новое состояние, после обновления фильтра Калмана.
- old_state — предыдущее состояние перед обновлением.
- K — Калмановский коэффициент.
- measurement — новое измерение (наблюдаемое значение).

- Обновление неопределенности. После того, как состояние обновлено, неопределенность уменьшается, поскольку мы уже учли новое измерение.

Формула в данной задаче:

$$\text{new_uncertainty} = (1 - K) \times \text{old_uncertainty}$$

Где:

- new_uncertainty — новая неопределенность после обновления.
- old_uncertainty — неопределенность до обновления.
- K — Калмановский коэффициент.

Основной код

Этот блок кода демонстрирует процесс симуляции движения объекта и использования фильтра Калмана для оценки его положения.

Начальные параметры

```
initial_position = 0  # Объект начинает с позиции 0
initial_uncertainty = 50  # Начальная неопределенность в нулевом положении
process_variance = 1e-2  # Шум процесса (например, небольшие ошибки модели)
measurement_variance = 0.5  # Шум измерений (более сильный шум, например, погрешность датчика)
velocity = 1  # Постоянная скорость объекта
```

-initial_position: Объект начинает с позиции 0.

-initial_uncertainty: Мы задаем высокую начальную неопределенность, что означает, что мы мало знаем о начальной позиции объекта.

-process_variance: Шум процесса небольшой, что говорит о том, что модель движения объекта будет относительно точна.

-measurement_variance: Шум измерений имитирует погрешность датчиков.

Создание объекта фильтра

```
# Создаем объект фильтра Калмана с заданными начальными параметрами.
kf = SimpleKalmanFilter(initial_position, initial_uncertainty, process_variance, measurement_variance)
```

Подготовка к симуляции

```
num_iterations = 40  # Количество шагов
true_positions = []  # Истинные позиции объекта
measured_positions = []  # Шумные измеренные позиции
estimated_positions = []  # Оцененные позиции, полученные через фильтр Калмана
true_position = initial_position  # Начальное истинное положение
```

-num_iterations: Число шагов симуляции.

-true_positions: Массив для хранения истинных положений объекта на каждом шаге.

-measured_positions: Массив для хранения измеренных (с шумом) положений объекта.

-estimated_positions: Массив для хранения оценок положения, полученных фильтром Калмана.

Основной цикл симуляции

```
for i in range(num_iterations):
    # Истинное положение объекта увеличивается с постоянной скоростью
    true_position += velocity
    true_positions.append(true_position)

    # Измерение положения с шумом (например, погрешность в работе датчика)
    measurement = true_position + np.random.normal(0, np.sqrt(measurement_variance))
    measured_positions.append(measurement)

    # Шаг предсказания
    kf.predict(velocity)

    # Шаг обновления (используем измерение для корректировки предсказания)
    estimated_position, uncertainty = kf.update(measurement)
    estimated_positions.append(estimated_position)
```

Здесь на каждом шаге цикла:

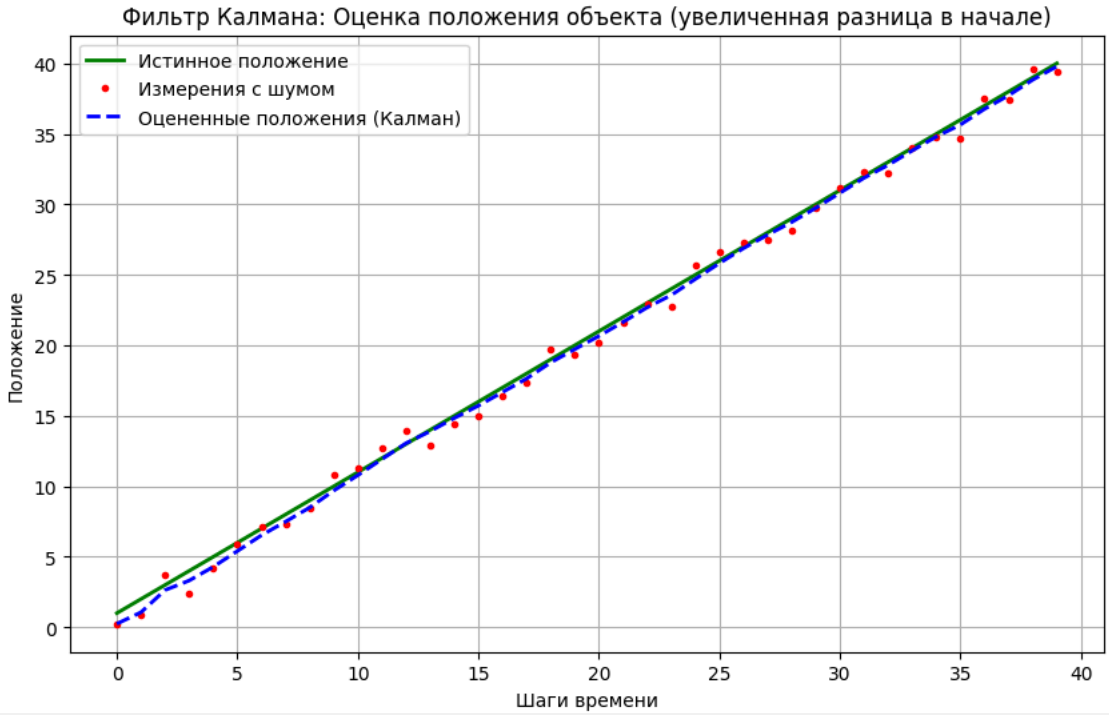
- Истинное положение увеличивается на значение скорости.
- Шумное измерение создается с помощью функции np.random.normal, которая добавляет нормальный шум с дисперсией, равной measurement_variance.
- После получения измерения выполняются:
  - Предсказание следующего положения объекта.
  - Обновление оценки с учетом измерения.
- Все данные сохраняются в соответствующие массивы для последующего построения графика.

Визуализация данных на графике

```
plt.figure(figsize=(10, 6))
time_steps = np.arange(num_iterations)

plt.plot(time_steps, true_positions, 'g-', label='Истинное положение', linewidth=2)
plt.plot(time_steps, measured_positions, 'r.', label='Измерения с шумом', markersize=6)
plt.plot(time_steps, estimated_positions, 'b--', label='Оцененные положения (Калман)', linewidth=2)

plt.title('Фильтр Калмана: Оценка положения объекта (увеличенная разница в начале)')
plt.xlabel('Шаги времени')
plt.ylabel('Положение')
plt.legend()
plt.grid()
plt.show()
```


Фильтр Калмана: Оценка положения объекта (увеличенная разница в начале)
Истинное положение
Измерения с шумом
Оцененные положения (Калман)
Положение
Шаги времени
0
5
10
15
20
25
30
35
40